

Capstone Project: Texas Registry Search Database

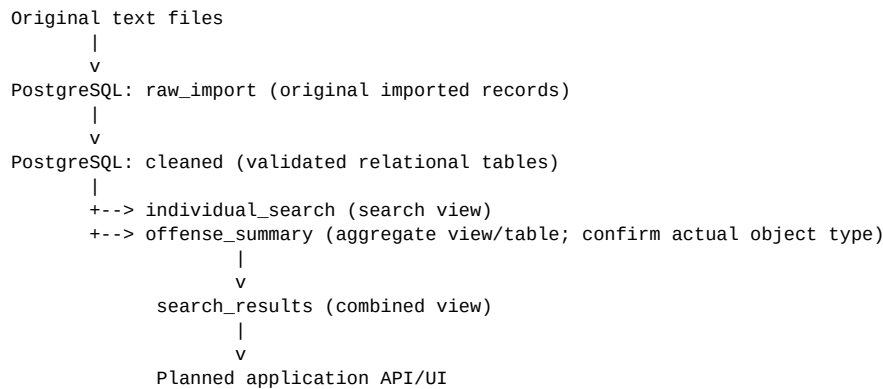
Project status

Database foundation implemented; final verification and application development remain. This document reflects the PostgreSQL results observed during development on October 3, 2026. It does not claim that the entire source dataset has been cleaned or that an application has been completed.

Objective

Prepare a relational PostgreSQL database from the supplied Texas registry text dataset to support an application that searches individuals and retrieves related records. Application features and technology stack beyond PostgreSQL are still to be finalized by the team.

Data architecture



The `raw_import` schema retains imported source records. The `cleaned` schema contains the working relational model. Preserve the raw schema so cleaning and transformation decisions can be audited.

Core relational model

Cleaned table	Primary key	Relationship
individual	ind_idn	Parent individual record
person	per_idn	ind_idn references individual.ind_idn
name	nam_idn	per_idn references person.per_idn
address	addressid	ind_idn references individual.ind_idn
offense	offense_record_id	ind_idn references individual.ind_idn

The primary and foreign key constraints above were verified using PostgreSQL catalog queries. The original `offenseid` is **not** unique and must not be treated as the cleaned table's primary key.

Data cleaning and verification

Offense deduplication

- Imported offense records: **161,468**

- Exact duplicate rows: **17**
- Cleaned offense records: **161,451**
- Distinct original offenseid values in the cleaned table: **161,360**
- Distinct cleaned offense_record_id values: **161,451**

Some records share an offenseid but differ in other fields, including aov_nbr or sov_cod. Only exact duplicate rows were removed. A surrogate primary key (offense_record_id) preserves the remaining distinct rows. The meaning of these variations should be confirmed against the dataset documentation before making further deduplication decisions.

Referential integrity

- All **161,451** cleaned offense rows matched an individual in the imported individual table.
- A validation query found **1** raw address row without a corresponding record in cleaned.individual (**107,866** matched of **107,867** raw addresses).
- All **108,219** raw person records matched cleaned.individual.
- All **396,705** raw name records matched a raw person record.

The unmatched address needs a documented handling decision. Do not silently invent an individual or discard the raw record.

Field validation and conversion

- Validation checks reported **0 invalid nonblank numeric values** for address.lat_nbr, address.lon_nbr, person.hgt_qty and person.wgt_qty.
- All **161,451** cleaned offense records had populated cdd_dte values; the date-format validation found **0 unexpected formats**.
- cleaned.offense.cdd_dte was confirmed as PostgreSQL timestamp without time zone after conversion.
- Other listed cleaned source columns were observed as text at the time of schema inspection. Further typed conversions are optional future work and must be tested first.

Search layer

The current database contains these search objects:

- cleaned.individual_search: joins individual, person and name information for name-based search.
- cleaned.offense_summary: aggregates offense information by individual. Its existence and relationship check were verified; confirm its object type and precise SQL definition when exporting the schema.
- cleaned.search_results: joins the searchable individual/name records with offense summaries, preserving records with no offense summary via a left join.

Observed counts

Check	Result
cleaned.individual_search rows	396,891
Distinct individuals in individual_search	108,405
cleaned.offense_summary rows	108,152
Unmatched offense summary individuals	0
cleaned.search_results rows	396,891

Check	Result
Distinct individuals in search_results	108,405
Search/name rows with total_offense_records > 0	396,502
Search/name rows without offense records	389

Important: Search rows are not unique individuals: people may have multiple name records. Do not interpret 396,502 as a count of distinct people with offenses. The number of search rows (396,891) also exceeds the raw name table count (396,705); verify join cardinality and confirm that this is intentional before release.

Useful validation queries

```
-- Search layer counts
SELECT COUNT(*) AS total_rows,
       COUNT(DISTINCT ind_idn) AS unique_individuals,
       COUNT(*) FILTER (WHERE total_offense_records > 0) AS rows_with_offenses
FROM cleaned.search_results;

-- Confirm every offense summary links to an individual
SELECT COUNT(*) AS summary_records,
       COUNT(*) FILTER (WHERE i.ind_idn IS NULL) AS unmatched_individuals
FROM cleaned.offense_summary s
LEFT JOIN cleaned.individual i ON s.ind_idn = i.ind_idn;

-- Inspect key constraints
SELECT table_name, constraint_name, constraint_type
FROM information_schema.table_constraints
WHERE table_schema = 'cleaned'
  AND constraint_type IN ('PRIMARY KEY', 'FOREIGN KEY')
ORDER BY table_name, constraint_type;

-- Inspect a small sample of name searches
SELECT ind_idn, nam_txt, total_offense_records, unique_offenses,
       earliest_offense_date, latest_offense_date
FROM cleaned.search_results
WHERE lna_txt ILIKE 'SMITH'
LIMIT 10;
```

The last query assumes the previously discussed search view exposes the listed columns. Confirm its definition before adopting it as an application query.

Remaining work

- 1. Final database QA:** Check search results, join cardinality, offense-summary accuracy, alias handling and the unmatched address; review performance using EXPLAIN (ANALYZE, BUFFERS) on representative queries.
- 2. Schema reproducibility:** Export a reviewed, schema-only SQL file or write ordered migrations for the cleaned tables, keys, indexes and search objects. Do not commit raw records.
- 3. Backend:** Select an application stack, connect to PostgreSQL with least-privilege credentials, implement parameterized search queries, pagination, input validation and error handling.
- 4. Frontend:** Build the team-approved search interface and results/detail views; determine whether geographic filtering or other features are in scope.
- 5. Integration/security/testing:** Verify end-to-end behavior, query speed, access controls, logging and appropriate handling of sensitive personal data.
- 6. Capstone deliverables:** Produce an ER diagram, cleaning methodology, reproducible setup instructions, test results and a demo/presentation aligned with the course rubric.

Repository layout (proposed)

```
capstone-project/
■■■ README.md
■■■ .gitignore
■■■ docs/           # ER diagram, cleaning log, test notes
■■■ sql/            # Reviewed schema-only DDL, migrations, safe queries
■■■ backend/        # Planned API code
■■■ frontend/       # Planned user interface
```

The current README documents observed work; it is **not** a complete, reproducible installation procedure. Schema DDL, exact import steps, application stack and dependencies still need to be exported or agreed upon.

Data safety

- Do **not** commit the original registry ZIP, extracted text files, database dumps containing records, .env files, passwords or connection strings.
- Prefer a private repository while the team decides what can be published.
- Use synthetic fixtures for development and testing. Public availability of source data does not automatically make republishing the entire dataset necessary or appropriate.
- If sensitive data is accidentally committed, removing it in a later commit is insufficient: revoke exposed credentials and follow a history-removal and incident-response process.